

Chapitre 2 : Listes en OCaml

I Théorie et définition inductive

En programmation fonctionnelle, la liste est une structure de données fondamentale, homogène (tous les éléments ont le même type) et de taille dynamique. Contrairement aux tableaux, les listes ne sont pas indexées en mémoire de manière contiguë.

Définition : Une liste en OCaml est définie de manière **inductive** (ou récursive) :

- Soit la liste est vide, notée `[]`.
- Soit elle est composée d'un élément de tête x (de type `'a`) et d'une suite d'éléments appelée reste ou queue xs (de type `'a list`). On la note alors $x :: xs$.

L'opérateur `::` (prononcé *Cons*) permet de construire une liste en ajoutant un élément en tête. Il est associatif à droite.

Exemple : La liste contenant les entiers 1, 2 et 3 peut s'écrire explicitement `[1; 2; 3]` ou de manière équivalente `1 :: 2 :: 3 :: []`.

Attention Ne confondez pas le séparateur d'éléments dans une liste qui est le point-virgule (`;`) avec la virgule (`,`), qui sert à définir des n-uplets (tuples, nous reverrons cela dans le prochain chapitre).

II Analyse de complexité et représentation mémoire

En mémoire, une liste est une chaîne de pointeurs (une liste simplement chaînée). Chaque maillon contient une valeur et un pointeur vers le maillon suivant. Cette implémentation dicte directement la complexité des opérations de base sur les listes :

- **Ajout en tête (`::`) :** Cette opération se contente de créer un nouveau maillon pointant vers l'ancienne liste. C'est une opération en temps constant, soit en $\Theta(1)$.
- **Accès à la tête :** Immédiat, en $\Theta(1)$.
- **Accès au n -ième élément ou calcul de la taille :** Nécessite de parcourir la liste maillon par maillon. Pour une liste de taille N , cela s'effectue en $\Theta(N)$.

Remarque : Puisque les structures de données en programmation fonctionnelle pure sont immuables, l'ajout en tête ne copie pas la liste existante ; elle est partagée en mémoire (*structural sharing*), ce qui rend le `::` extrêmement efficace.

III Le module `List`

Vocabulaire : En OCaml, un **module** regroupe un ensemble de types et de fonctions liés à une même thématique. Le module `List` de la bibliothèque standard fournit les fonctions indispensables pour manipuler les listes sans avoir à les réécrire.

Pour appeler une fonction d'un module, on utilise la notation pointée : `Nom_du_module.nom_fonction`.

A Fonctions de base du module `List`

Voici les fonctions incontournables, leurs signatures, leurs rôles et leurs complexités.

1 `List.hd` et `List.tl`

- **Signatures** : `List.hd` : `'a list -> 'a` et `List.tl` : `'a list -> 'a list`
- **Rôle** : `hd` (*head*) renvoie le premier élément ; `tl` (*tail*) renvoie la liste privée de son premier élément.
- **Complexité** : $\Theta(1)$ pour les deux opérations.

✗ **Attention** ✗ Ces deux fonctions lèvent une exception `Failure` si la liste passée en argument est vide. Il est souvent préférable d'utiliser le pattern matching.

2 `List.length`

- **Signature** : `List.length` : `'a list -> int`
- **Rôle** : Renvoie le nombre d'éléments de la liste.
- **Complexité** : $\Theta(N)$ où N est la longueur de la liste.

3 `List.append` ou opérateur `@`

- **Signature** : `List.append` : `'a list -> 'a list -> 'a list`
- **Rôle** : Concatène deux listes pour n'en former qu'une seule.
- **Complexité** : $\Theta(N)$ où N est la taille de la **première** liste uniquement (car elle doit être parcourue et recréée pour pointer sur la deuxième).

B Théorie du Filtrage par Motif (Pattern Matching)**Méthode : Le mécanisme de filtrage**

Le `match ... with` permet d'évaluer une expression en la comparant successivement à une série de motifs (structures). Dès qu'un motif correspond, la branche associée est exécutée. En OCaml, le compilateur vérifie l'exhaustivité (tous les cas possibles sont traités) et la redondance (aucun motif n'est inutile).

❗ **Remarque** : Le motif `head::tail` sépare de manière sûre la tête du reste de la liste, évitant ainsi l'utilisation de fonctions partielles comme `List.hd` ou `List.tl` qui peuvent lever des exceptions sur une liste vide.

💡 **Exemple** : Voici un exemple de fonction utilisant le filtrage par motif pour calculer la somme des éléments d'une liste d'entiers :

```
1 let rec sum_list lst =
2   match lst with
3   | [] -> 0
4   | head :: tail -> head + sum_list tail
```

❗ **Remarque** : Le *pattern matching* met en évidence l'importance de la récursion en programmation fonctionnelle, permettant de traiter des structures de données inductives comme les listes de manière élégante et sûre.

C Le Filtrage Simultané (Double Match)**Méthode : Filtrage de plusieurs valeurs**

Il est possible de filtrer plusieurs expressions simultanément en les regroupant dans un n-uplet (couple, triplet, etc.) séparé par des virgules : `match expr1, expr2 with`. C'est la méthode optimale pour traiter deux listes en parallèle.

💡 **Exemple :** Voici comment réaliser l'intersection de deux listes triées en une seule passe ($\Theta(N + M)$) grâce au double matching :

```
1 let rec inter_sorted l1 l2 =
2   match l1, l2 with
3   | [], _ -> []
4   | _, [] -> []
5   | head1 :: tail1, head2 :: tail2 ->
6     if head1 < head2 then inter_sorted tail1 l2
7     else if head1 > head2 then inter_sorted l1 tail2
8     else head1 :: inter_sorted tail1 tail2
```

✗ Attention ✗ L'ordre des motifs compte. OCaml examine les lignes du haut vers le bas. Les cas généraux ou terminaux (comme `[]`, `_`) doivent être placés avant le cas structurel `head :: tail` pour intercepter correctement les listes vides.

IV Fonctions d'ordre supérieur

Le module `List` brille par ses fonctions prenant d'autres fonctions en paramètres (fonctions d'ordre supérieur).

1 List.map

Applique une fonction f à chaque élément $[x_1; x_2; \dots; x_n]$ pour retourner $[f(x_1); f(x_2); \dots; f(x_n)]$.

- **Signature :** `List.map : ('a -> 'b) -> 'a list -> 'b list`
- **Complexité :** $\Theta(N)$.

```
1 (* Exemple d'utilisation : doubler les elements d'une liste *)
2 let doubler_liste l = List.map (fun x -> x * 2) l
```

2 List.filter

Retourne une nouvelle liste ne contenant que les éléments qui valident un prédicat (une fonction retournant un booléen).

- **Signature** : `List.filter : ('a -> bool) -> 'a list -> 'a list`
- **Complexité** : $\Theta(N)$.

```
1 (* Exemple d'utilisation : ne garder que les entiers pairs *)
2 let garder_pairs l = List.filter (fun x -> x mod 2 = 0) l
```

 **Application :** Écrire une fonction récursive appartient : 'a -> 'a list -> bool qui vérifie la présence d'un élément dans une liste sans utiliser le module List. Donner sa complexité dans le pire des cas.

[illegible]

Contents

I	Théorie et définition inductive	1
II	Analyse de complexité et représentation mémoire	1
III	Le module <code>List</code>	1
A	Fonctions de base du module <code>List</code>	1
1	<code>List.hd</code> et <code>List.tl</code>	2
2	<code>List.length</code>	2
3	<code>List.append</code> ou opérateur <code>@</code>	2
B	Théorie du Filtrage par Motif (Pattern Matching)	2
C	Le Filtrage Simultané (Double Match)	2
IV	Fonctions d'ordre supérieur	3
1	<code>List.map</code>	3
2	<code>List.filter</code>	3

Crédits

Ce cours est mis à disposition selon les termes de la Licence Creative Commons  **Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 4.0 International**. Pour voir une copie de cette licence, visitez <http://creativecommons.org/licenses/by-nc-sa/4.0/>.